

Computer Architecture: The Debugger & Embedded Systems

Dr. Colin O'Flynn

Objective of Today's Lecture

- What code did I write?
- What code did the compiler generate?
- How do we observe it running?

..This will lead into the first lab

Learning Objectives

- What a debugger **can** and **cannot** know
- Understand how **debug symbols** map source code to machine code
- Observe and understand how **optimizations change debugging**

What is a bug?

92

9/9

0800 Antan started
1000 " stopped - antan ✓

1300 (032) MP-MC { 1.2700 · 9.037 847 025
1.982647000 9.037 846 995 correct
2.130476415 (2) 4.615925059(-2)
(033) PRO 2 2.130476415
correct 2.130676415

Relays 6-2 in 033 failed special speed test
in relay " 10.000 test.

Relays changed

1700 Started Cosine Tape (Sine check)
1525 Started Multi-Adder Test.

1545



Relay #70 Panel F
(moth) in relay.

First actual case of bug being found.

1650 Antan started.
1700 closed down.

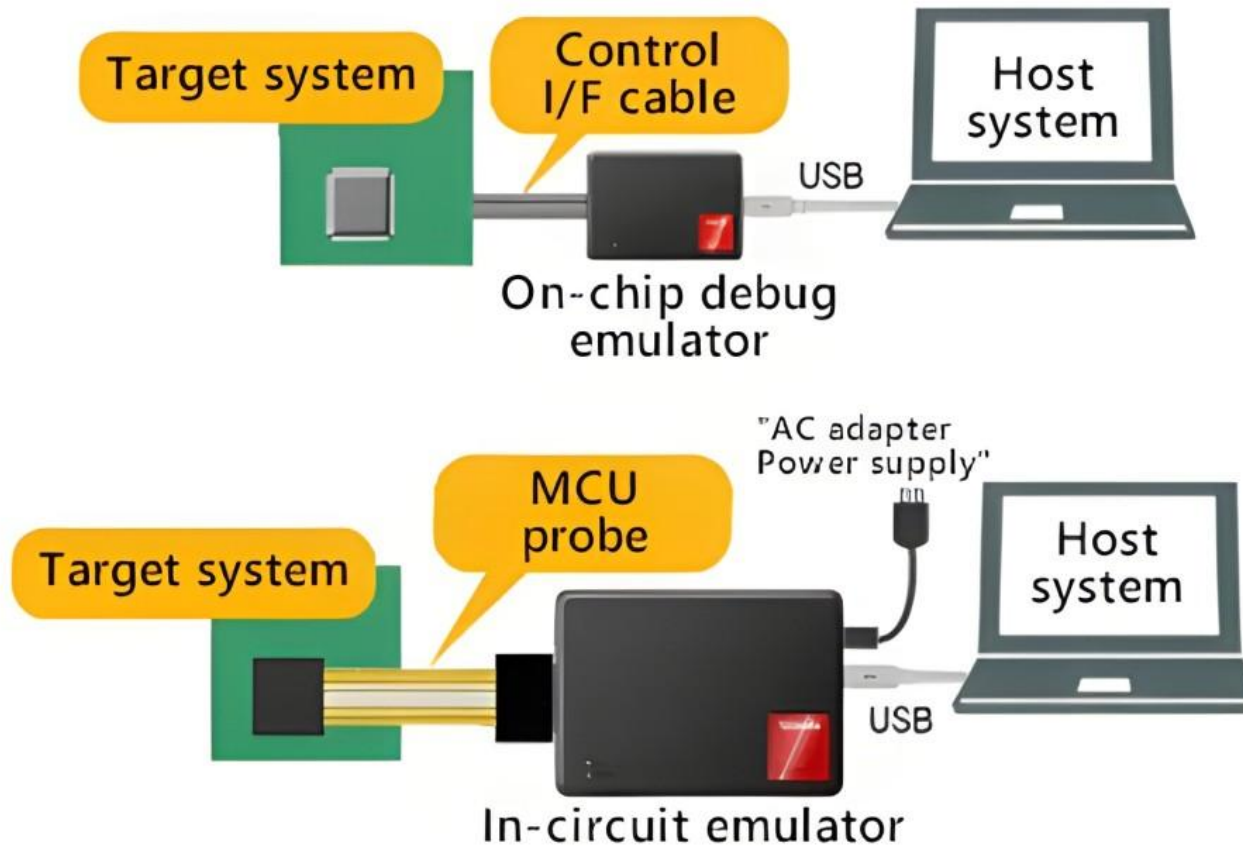
Relay 2745
Relay 3370

What is a debugger?

Debuggers & Emulators History



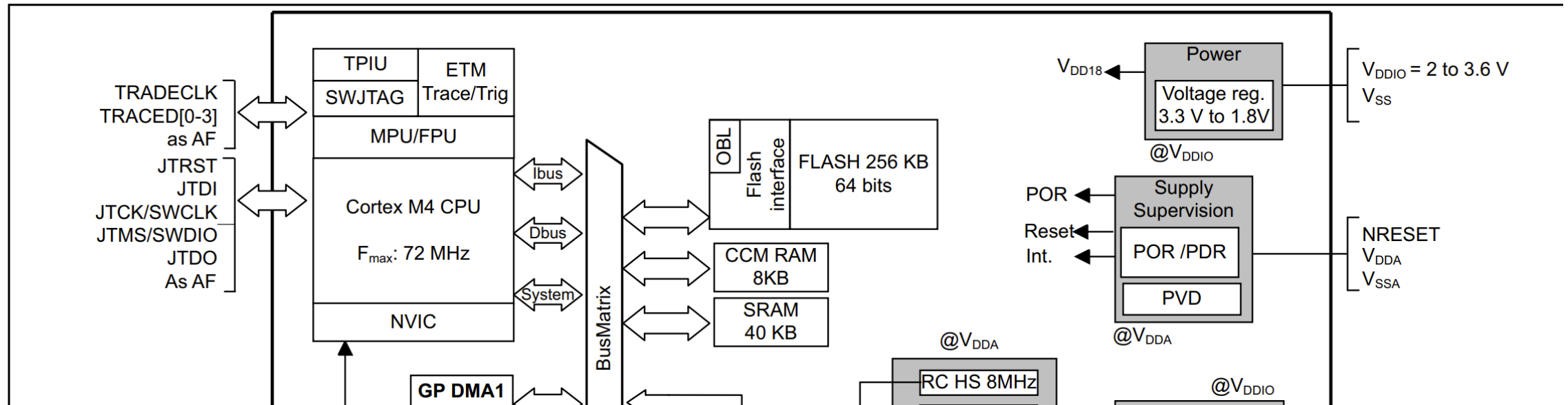
Debuggers: In Circuit Emulators & On-Chip Debug



Cortex-M In-Circuit Debugger

- 99% of chips you use will have **On-Chip Debug** support

Figure 1. STM32F303xB/STM32F303xC block diagram

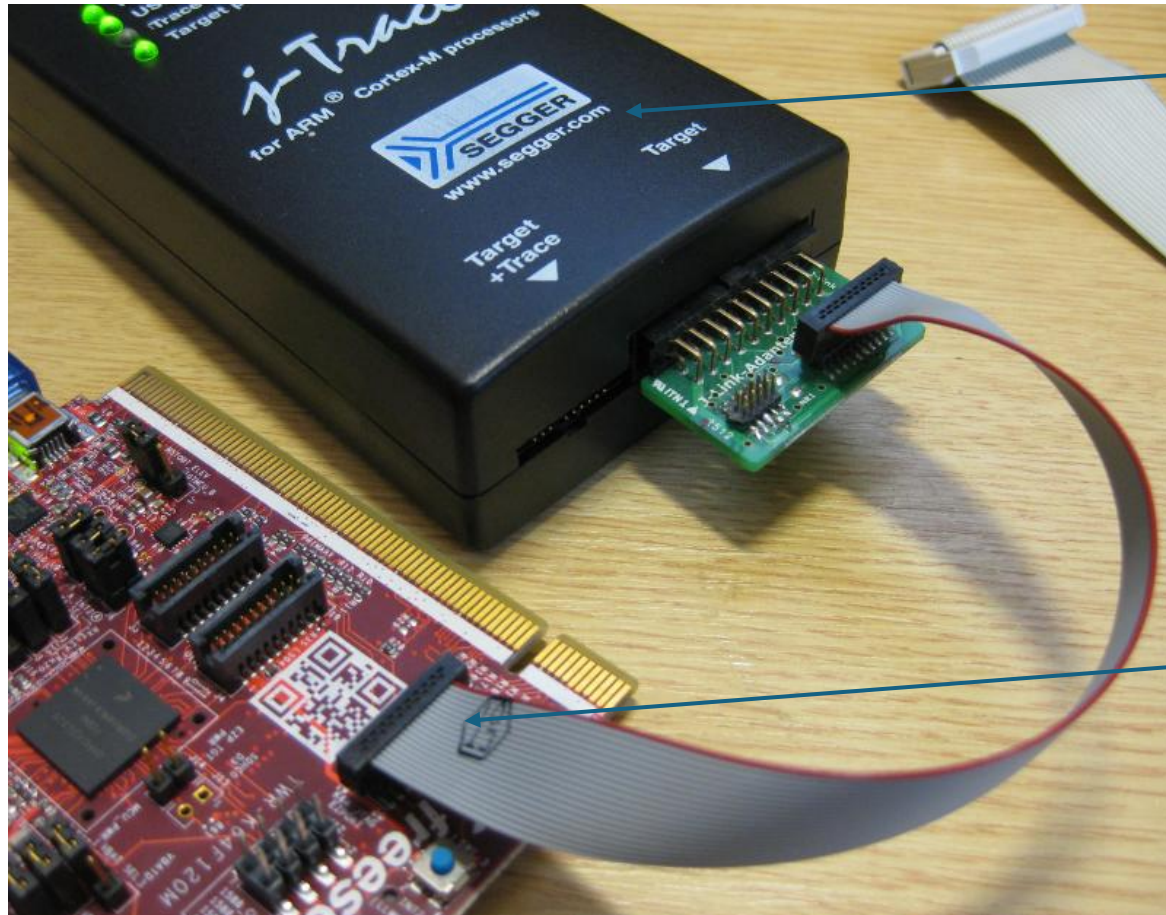


JTAG? SWD?

- JTAG is a *generic* interface for connecting to devices for testing & debug.
- A JTAG connection *may* connect to a debug “core” if the chip has one.
- Arm devices may support a “smaller pin count” version of JTAG called *Serial Wire Debug (SWD)*



Debugger Connection to System



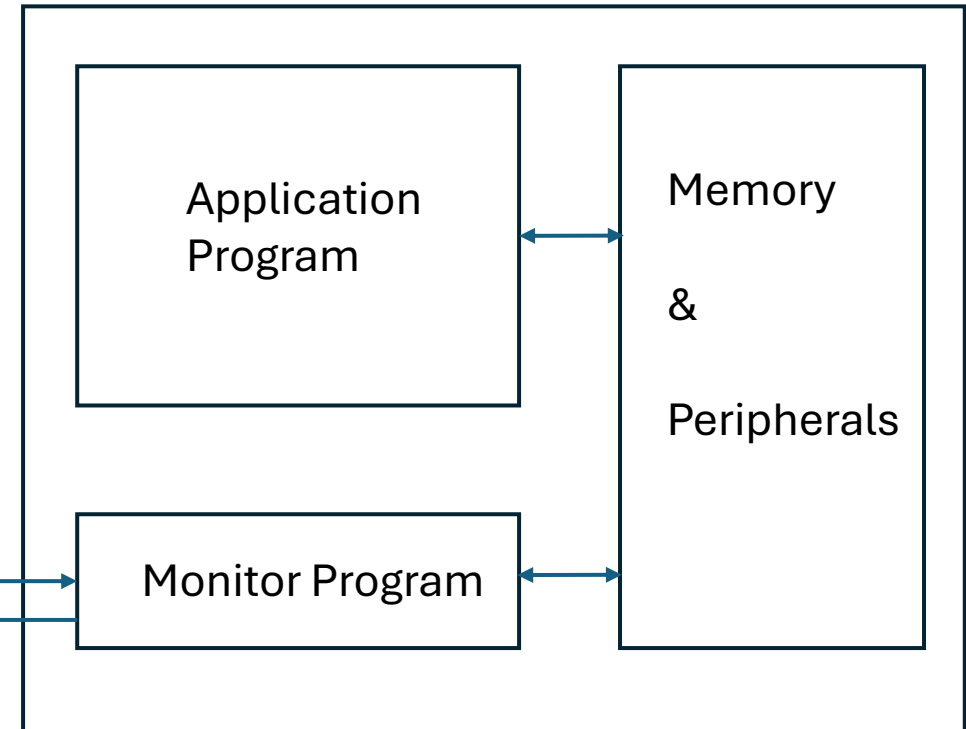
Debugger

Target System

Debug Monitors



```
$ read_mem 0x2001000, 16  
00 00 01 FA FE 0B 0C 0D  
AB 02 03 04 1E 00 20 22
```



Debug Monitors

- Less common now – these existed mainly when on-chip debuggers were not prevalent
- The *naming* remains, we will often see references to a “monitor”
- Monitors could break, step code, etc similar to a debugger

Debuggers & Source Code

```
C++ source #1
1  #include <stdio.h>
2
3  void gpio_on(void);
4  void gpio_off(void);
5  void gpio_toggle();
6
7  void toggle_led(void){
8      //gpio_on();
9      //gpio_off();
10     gpio_toggle();
11 }
12
13 int some_function(int data){
14     if (data > 0) {
15         for(int i = 0; i < 4; i++){
16             toggle_led();
17         }
18         return 0;
19     }
20
21     if (data < 0) {
22         for(int i = 0; i < 10; i++){
23             toggle_led();
24         }
25     }
26     if (data > 0){
27         toggle_led();
28         return 2;
29     }
30     return 3;
31 }
```

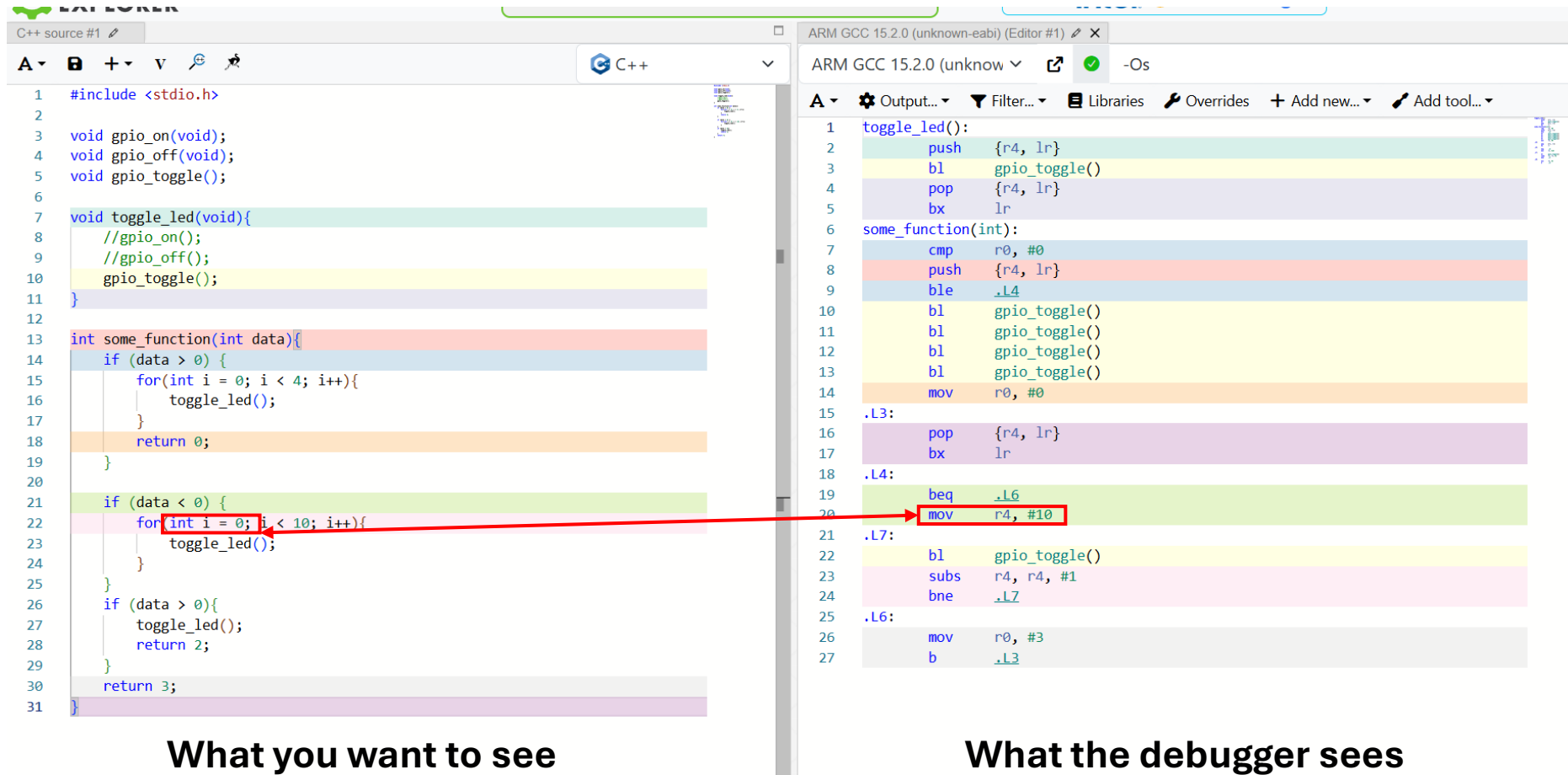
What you want to see

```
ARM GCC 15.2.0 (unknown-eabi) (Editor #1)
ARM GCC 15.2.0 (unknown -Os
A Output... Filter... Libraries Overrides + Add new... Add tool...
1 toggle_led():
2     push    {r4, lr}
3     bl      gpio_toggle()
4     pop     {r4, lr}
5     bx      lr
6 some_function(int):
7     cmp     r0, #0
8     push    {r4, lr}
9     ble     .L4
10    bl      gpio_toggle()
11    bl      gpio_toggle()
12    bl      gpio_toggle()
13    bl      gpio_toggle()
14    mov     r0, #0
15 .L3:
16    pop     {r4, lr}
17    bx      lr
18 .L4:
19    beq     .L6
20    mov     r4, #10
21 .L7:
22    bl      gpio_toggle()
23    subs    r4, r4, #1
24    bne     .L7
25 .L6:
26    mov     r0, #3
27    b       .L3
```

What the debugger sees

(Debugger is also missing function names)

Debuggers & Source Code



What you want to see

What the debugger sees

(Debugger is also missing function names)

Debug Symbols

- Map function names, static variables, etc to addresses:

```
0040067c T putc
00400698 T getch
004006bc T trigger_setup
004006dc T trigger_low
004006fc T trigger_high
0040071c T uart_init
00400750 T uart_enable_tx
00400756 T uart_enable_rx
0040075c T uart_write
0040076a T uart_read
0040077a T pio_set_peripheral
004007e8 T pio_set_input
00400814 T pio_set_output
00400838 T pio_set_pin_high
00400852 T pio_set_pin_low
0040086c T pio_configure_pin
00400918 T SystemCoreClockUpdate
004009e4 T system_init_flash
00400a48 T pmc_switch_mck_to_sclk
00400a88 T pmc_switch_mck_to_mainclk
00400acc T pmc_switch_mainclk_to_xtal
00400b18 T pmc_osc_is_ready_mainclk
00400b28 T pmc_enable_periph_clk
00400b68 T pmc_set_unitarytest
```

NOTE: This *doesn't* include optimized away variables...

Debugger Tool #1: Breakpoints

- Breakpoints cause execution to “break” (stop)
- Control is passed to the debug core / tool at this point

Code & Data Breakpoints

- You set a match on some condition that causes the break
- Most commonly we break on a program location (code breakpoint)
- You can also break on data read/write ("data breakpoint" or "watchpoint")

The Program Counter

- Will return to this when we talk more about CPU cores, but quickly:

The image shows a debugger window with two panes. The left pane displays assembly code for a program named `main.c`. The right pane shows the state of the CPU registers.

Assembly Code Pane:

```
08000E24 .word 0x08000E24
— main.c — 1 —
#include <stdio.h>
int main(void){
printf("Hello World");
4801 ldr r0, =0x08000E34
F000FC82 bl 0x08000AFE <printf>
— main.c — 5 —
while(1);
E7FE b 0x080001FA
08000F34 .word 0x08000F34
```

The instruction `ldr r0, =0x08000E34` at address `080001f4` is highlighted in green. A yellow arrow points to the address `080001f4` in the left margin. A red arrow points from the `pc(r15)` register value in the right pane to the address `080001f4` in the left pane.

CPU Registers Pane:

Register	Value
r0	0x0800020b
r1	0x08000e24
r2	0x100004d0
r3	0x08000209
r4	0x08000f44
r5	0x00000000
r6	0x00000000
r7	0x00000000
r8	0x00000000
r9	0x00000000
r10	0x00000000
r11	0x00000000
r12	0x00000000
sp(r13)	0x20003000
lr(r14)	0x0800020f
pc(r15)	0x080001f4
xpsr	0x01000000
MSP	0x20003000
PSP	0x00000000
PRIMASK	0x00
BASEPRI	0x00
FAULTMASK	0x00
CONTROL	0x00000000
internal	0x00000000

Assembly vs. C Stepping

The image displays two side-by-side debugger windows illustrating the relationship between assembly code and its corresponding C source code.

Left Window (Disassembly): This window shows the assembly code for the `main` function. The address `080001f4` is highlighted, corresponding to the instruction `ldr r0, =0x08000E34`. The assembly code includes comments for the source files: `main.c` and `SEGGER_THUMB_Startup.s`.

```
08000E24 .word 0x08000E24
— main.c — 1 —
#include <stdio.h>
int main(void){
printf("Hello World");
080001f4 4801 ldr r0, =0x08000E34
080001f6 F00FC82 bl 0x08000AFE <printf>
— main.c — 5 —
while(1);
080001fa E7FE b 0x080001FA
08000E34 .word 0x08000E34
— SEGGER_THUMB_Startup.s — 165 —
// * Perform segment init
// * Perform heap init (if used)
// * Call constructors of global Objects (if any exist)
//
```

Right Window (C Source): This window shows the C source code for the `main` function. The line `printf("Hello World");` is highlighted, corresponding to the assembly instruction at `080001f6` in the left window.

```
main.c stm32f303x8_Vectors.s core_cm4.h
int main0
1 #include <stdio.h>
2
3 int main(void){
4     printf("Hello World");
5     while(1);
6 }
```

e.g., Many source lines → Fewer Assembly Line

```
1  #define TEMP_BASE 23
2  int test_function(int a, int b){
3      int result;
4
5      //Add 7 mV offset frp, datasheet
6      a = a + 7;
7
8      //Remove base temp offset
9      a = a - TEMP_BASE;
10
11     // Add user offset
12     result = a + b;
13
14     //Return user offset
15     return result;
16 }
```

```
1  test_function(int, int):
2      sub    r0, r0, #16
3      add    r0, r0, r1
4      bx     lr
```

e.g., One Source Line → Many Assembly Lines

```
1 int test_function(int a){  
2     return (a * 1.3) / 2.7 + 42;  
3  
4 }
```

```
1 test_function(int):  
2     push    {r4, lr}  
3     bl      __aeabi_i2d  
4     ldr     r2, .L3  
5     ldr     r3, .L3+4  
6     bl      __aeabi_dmul  
7     ldr     r2, .L3+8  
8     ldr     r3, .L3+12  
9     bl      __aeabi_ddiv  
10    mov     r2, #0  
11    ldr     r3, .L3+16  
12    bl      __aeabi_dadd  
13    bl      __aeabi_d2iz  
14    pop     {r4, lr}  
15    bx      lr  
16 .L3:  
17     .word  -858993459  
18     .word  1073007820  
19     .word  -1717986918  
20     .word  1074108825  
21     .word  1078263808
```

Register Your Trust

The screenshot shows an IDE with a C program and hardware monitoring windows. The code in `main.c` is as follows:

```
1 #include <stdio.h>
2
3 int main(void){
4     for(int i = 1; i < 10; i++){
5         printf("Hello World %d", i);
6     }
7     while(1);
8 }
```

The `printf` statement on line 5 is highlighted in green. The `Watch 1` window on the right displays the following data:

Expression	Value
<code>i</code>	<code>0x00000002</code>
<code>g</code>	<code>g symbol not found</code>

Below the watch window, the `Registers 1` window shows the state of the CPU registers:

Name	Value	Address
CPU		
<code>r0</code>	<code>0x0000000d</code>	
<code>r1</code>	<code>0x20002f5d</code>	
<code>r2</code>	<code>0x00000000</code>	
<code>r3</code>	<code>0x100000a8</code>	
<code>r4</code>	<code>0x00000002</code>	
<code>r5</code>	<code>0x00000000</code>	
<code>r6</code>	<code>0x00000000</code>	

Simple Loop Example

```
15  /******  
16  *  
17  *      main()  
18  *  
19  *      Function description  
20  *      Application entry point.  
21  */  
22  int main(void) {  
23      int i;  
24  
25      for (i = 0; i < 100; i++) {  
26          printf("Hello World %d!\n", i);  
27      }  
28      do {  
29          i++;  
30      } while (1);  
31  }  
32  
33  /****** End of file *****/  
34
```

Simple Loop Example

```

    */
int main(void) {
    08000188  B580    push {r7, lr}
    0800018a  B082    sub sp, sp, #8
    0800018c  2000    movs r0, #0
    0800018e  9001    str r0, [sp, #4]
    ————— main.c — 23 —————
    int i;
    for (i = 0; i < 100; i++) {
    08000190  9000    str r0, [sp]
    08000192  9800    ldr r0, [sp]
    08000194  2863    cmp r0, #0x63
    08000196  DC0A    bgt 0x080001AE
    ————— main.c — 26 —————
    printf("Hello World %d!\n", i);
    08000198  9900    ldr r1, [sp]
    0800019a  F6406087 movw r0, #0x0E87
    0800019e  F6C00000 movt r0, #0x0800
    080001a2  F000FD2F bl 0x08000C04 <printf>
    ————— main.c — 25 —————
    for (i = 0; i < 100; i++) {
    080001a6  9800    ldr r0, [sp]
    080001a8  3001    adds r0, #1
    080001aa  9000    str r0, [sp]
    080001ac  E7F1    b 0x08000192
    ————— main.c — 26 —————
    printf("Hello World %d!\n", i);
    }
    do {
    080001ae  9800    ldr r0, [sp]
    080001b0  3001    adds r0, #1
    080001b2  9000    str r0, [sp]
    ————— main.c — 30 —————
    } while (1);
    080001b4  2001    movs r0, #1
    080001b6  2800    cmp r0, #0
    080001b8  D1F9    bne 0x080001AE
    ————— main.c — 31 —————
    }
    080001ba  9801    ldr r0, [sp, #4]
    080001bc  B002    add sp, sp, #8
    080001be  BD80    pop {r7, pc}

```


SIDEBAR: Makefile vs. IDE Configuration

- Many systems will use a *Makefile* that has all the compiler options stored in it. You modify the *Makefile* to change build options (optimization levels etc).
- Some Integrated Development Environments (IDEs) allow you to configure this instead – the IDE builds the command based on options you select.

Simple Loop Example, Again (Optimized)

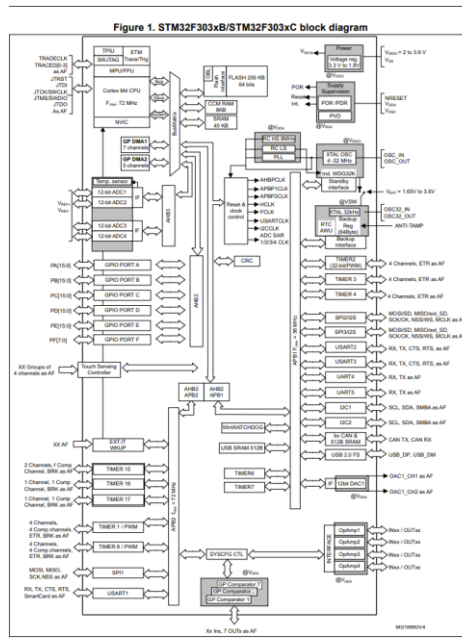
```

* Function description
* Application entry point.
*/
int main(void) {
    2400      movs r4, #0
    F64055CB  movw r5, #0x0DCB
    F6C00500  movt r5, #0x0800
    BF00      nop
    — main.c — 23 —
    int i;
    for (i = 0; i < 100; i++) {
    printf("Hello World %d!\n", i);
    4628      mov r0, r5
    4621      mov r1, r4
    F000FA4E  bl 0x08000B48 <printf>
    — main.c — 25 —
    for (i = 0; i < 100; i++) {
    3401      adds r4, #1
    2C64      cmp r4, #0x64
    D1F8      bne 0x080006A4
    BF00      nop
    — main.c — 26 —
    printf("Hello World %d!\n", i);
    }
    do {
    i++;
    } while (1);
    E7FE      b 0x080006B4
    — system_stm32f3xx.c — 171 —

```

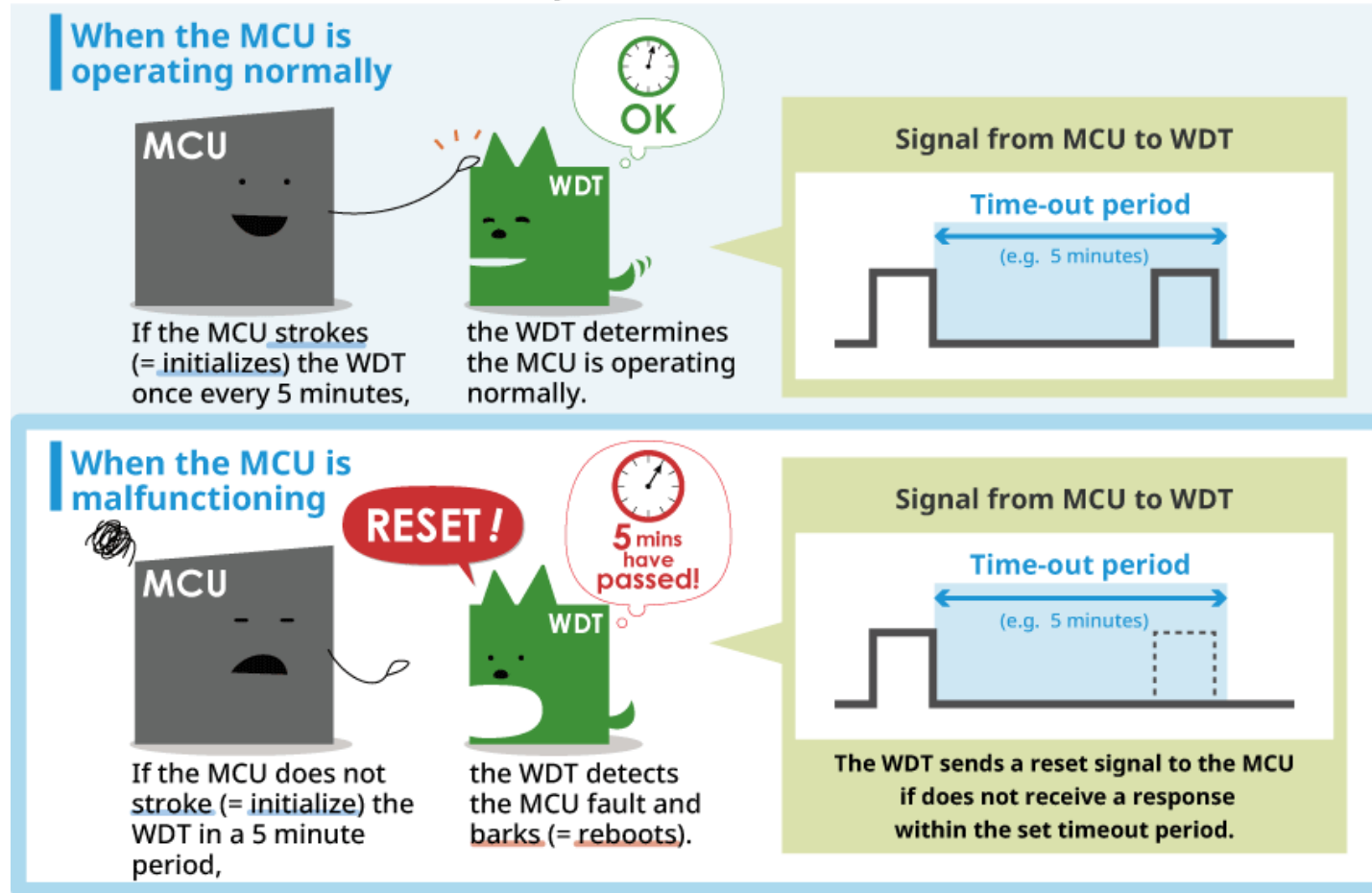
Debugger Side Effects

- The debugger is pausing your program
- The debugger is not pausing the entire world



Debuggers & Watchdog Timers (WDT)

● WDT operation (Time-out mode)



Debuggers & Watchdog Timers (WDT)

- WDT often have short timeouts (< 100 mS).
- If you pause the debugger for longer than that the WDT will reset the system.
- You may need to disable the WDT in the source code of your system for debugging.

CAVEATS:

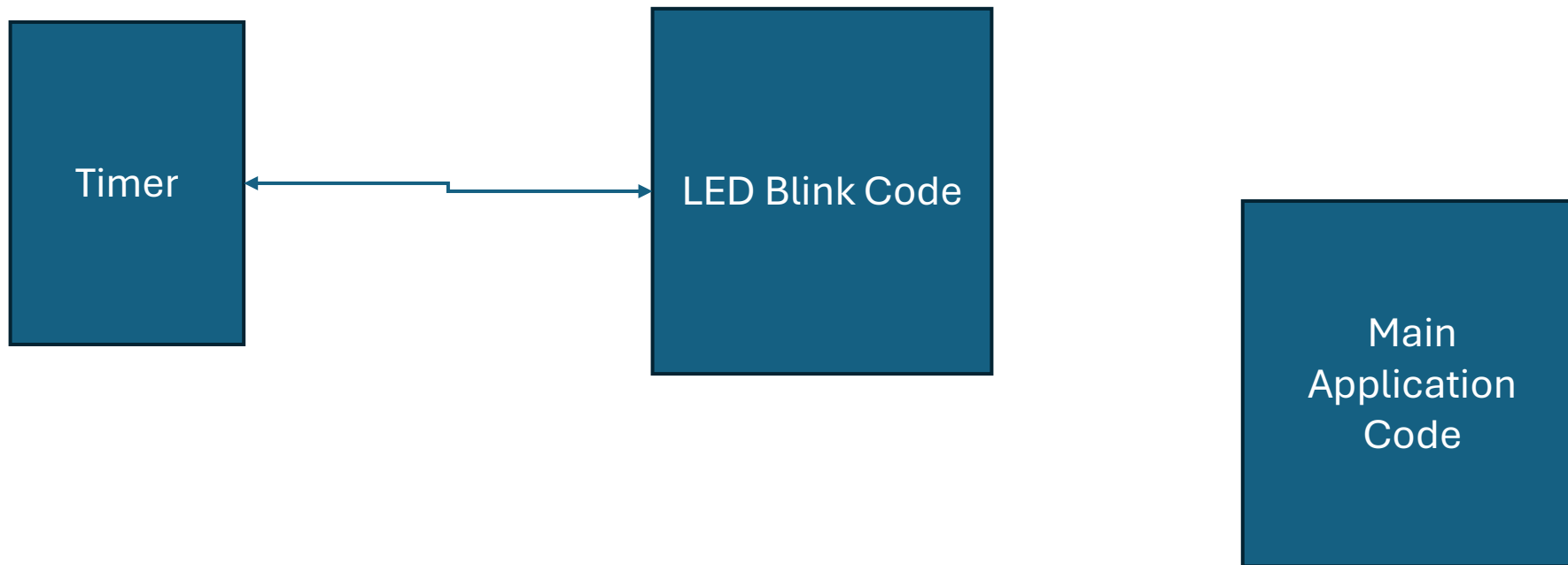
- Some debuggers have an option to automatically detect & do this.
- Some WDT built into microcontrollers have a way to pause during debug

Debuggers & Interrupts

- Interrupts often are time-sensitive – they interface to the outside world
- The debugger stops only the core from running – peripherals are often still working (generating interrupts), e.g.:

e.g., Timer Example

- You setup a timer to interrupt every 250 mS



e.g., Timer Example

- When attempting to step the main code, you will keep hitting the timer interrupt code
- **WHY: Every 250 mS the timer sets an interrupt flag. At which point the CPU jumps to the interrupt. So you are now continuously debugging the interrupt...**

Debuggers & UART

- Similar issues if your device is talking to an external board – the UART interrupt may fire on received data.
- You may need to modify your system for debugging!
- Some debuggers offer options to:
 - Pause timer peripherals
 - Automatically allow execution of interrupts

Debuggers & USB

- If you ever need to debug a USB device – this is very annoying, as the device will “time out” from Windows if it stops responding for more than a few seconds!
- Need to quickly pause & restart device...

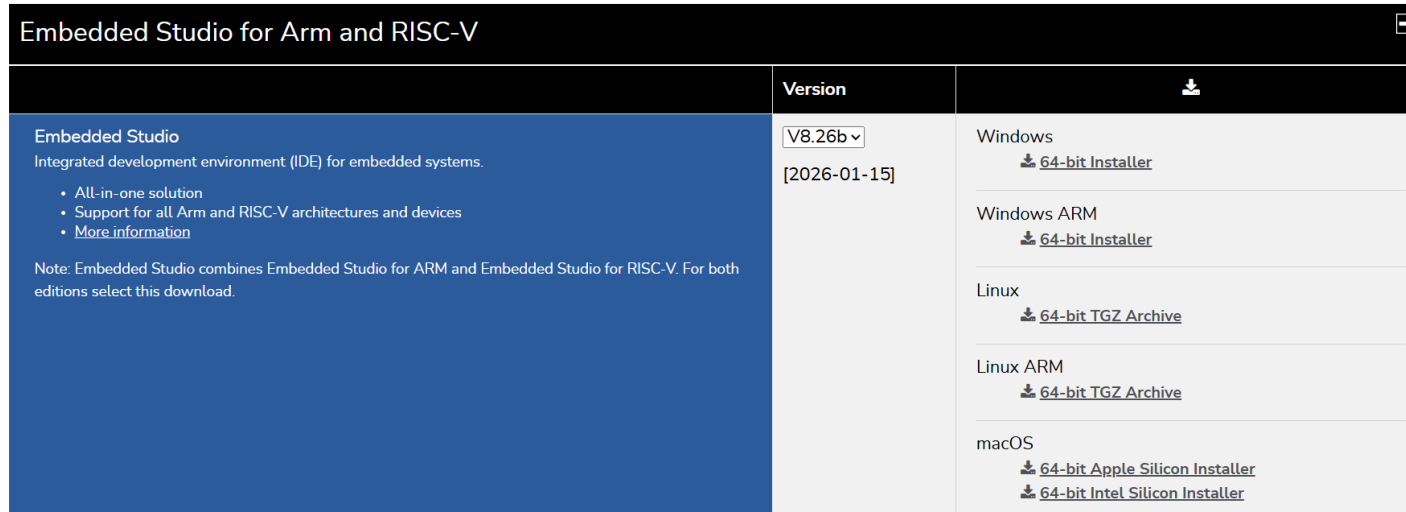
e.g., Triggering Crashes with Recursion

```
3
4 ☐ int callthisfunction(int n){
5     static char bigarray[200];
6     bigarray[0] += 1;
7
8     ☐ if (n == 0){
9         return 0;
10    }
11
12    callthisfunction(n - 1);
13
14    return bigarray[0];
15 }
16
17 ☐ /*******
18    *
19    *      main()
20    *
21    *  Function description
22    *  Application entry point.
23    */
24 ☐ int main(void) {
25     int i;
26
27     callthisfunction(10000);
28 }
```

Compiler + Debugger Install (Lab Prep)

We are going to use an Integrated Debug Environment (IDE) from Segger in this course:

<https://www.segger.com/downloads/embedded-studio/>



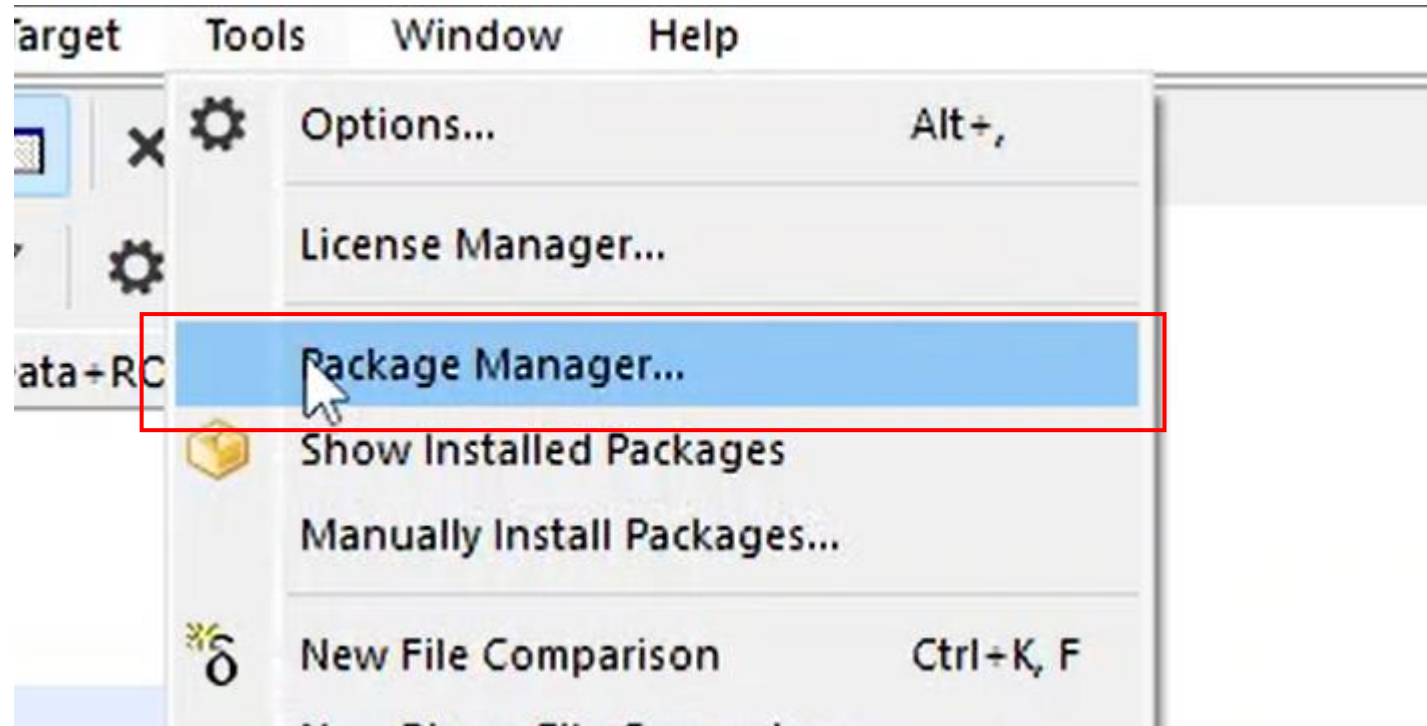
The screenshot shows the 'Embedded Studio for Arm and RISC-V' download page. It features a table with three columns: a description of the IDE, the version (V8.26b), and download links for various operating systems. The description highlights it as an 'All-in-one solution' for Arm and RISC-V. The version column shows 'V8.26b' and a date '[2026-01-15]'. The download column lists links for Windows (64-bit Installer), Windows ARM (64-bit Installer), Linux (64-bit TGZ Archive), Linux ARM (64-bit TGZ Archive), and macOS (64-bit Apple Silicon Installer and 64-bit Intel Silicon Installer).

	Version	Download
Embedded Studio Integrated development environment (IDE) for embedded systems. - All-in-one solution - Support for all Arm and RISC-V architectures and devices More information Note: Embedded Studio combines Embedded Studio for ARM and Embedded Studio for RISC-V. For both editions select this download.	V8.26b [2026-01-15]	Windows 64-bit Installer Windows ARM 64-bit Installer Linux 64-bit TGZ Archive Linux ARM 64-bit TGZ Archive macOS 64-bit Apple Silicon Installer 64-bit Intel Silicon Installer

Installation Work

1. Install SEGGER Embedded Studio.
2. You will be using this academically so can ACCEPT the license agreement (no license needed).
3. You will need to open the “Package Manager” (it may ask you this the first time) & get a list of packages

Installation Work (Cont'd)



 Select Packages

Title	Version	Type	Status	Action
-------	---------	------	--------	--------

SEGGER Embedded Studio V8.26b



Check For Packages

Do you want to download the latest list of packages now?

Yes

No



Show this dialog next time

Warning

No packages are visible because the **Internet Options > Automatically Check For Packages** environment option is set to **No**

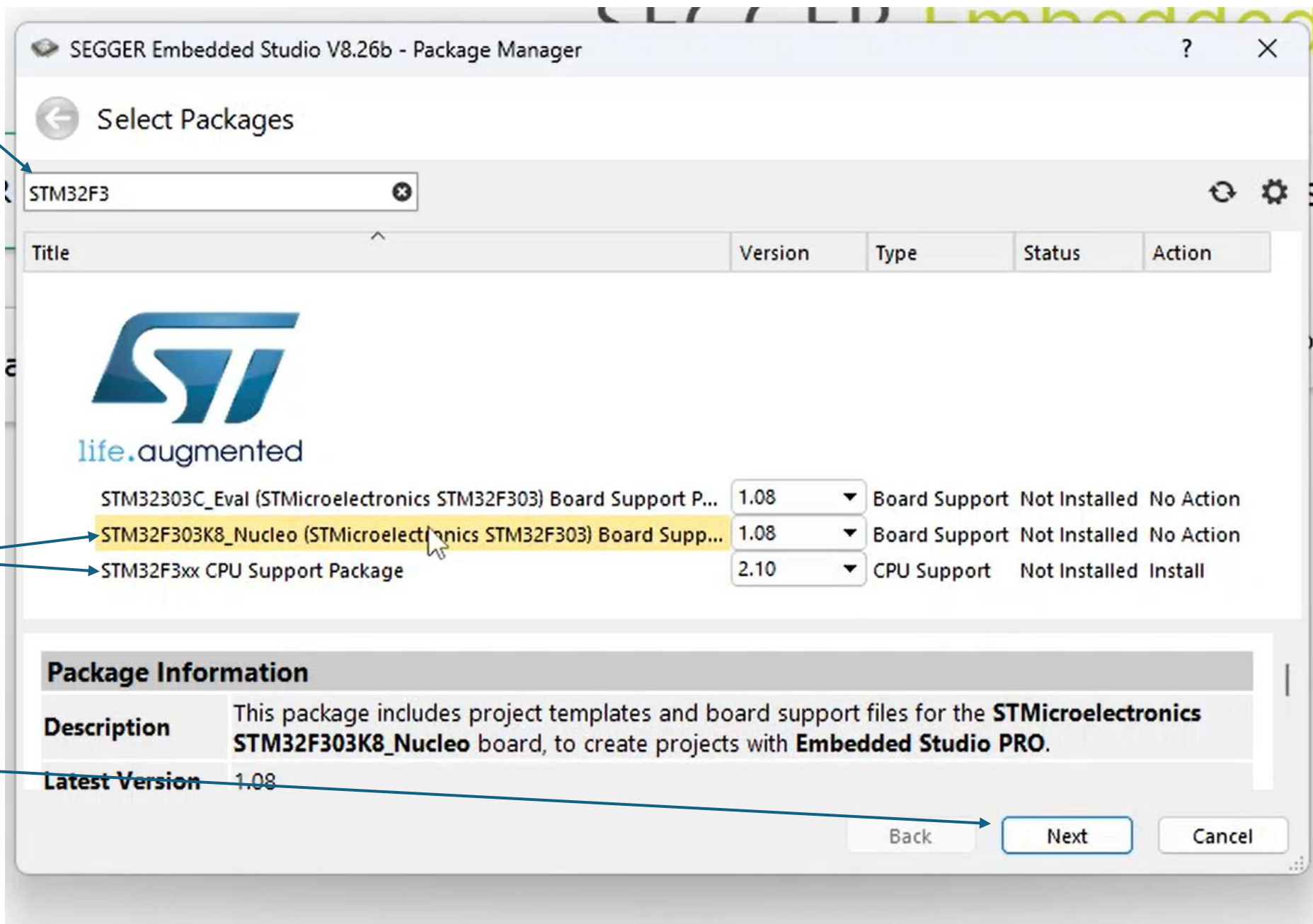
Back

Cancel

1. Search
STM32F3

2. Right click
on both of
these & click
"Install"

3. Click
"Next"



Ready for Lab #1

- Lab #1 can be done in small groups (1-3 people). Each person will check out a board as you may need to recreate parts of this for the assignment.
- We'll explore the debugger usage & see how changes in optimization affect code, as well as understand how to use various debugger features.
- The same setup will be used in the remaining labs in the course.